User requirements

1. Employee at the bunk should be able to input the sale
2. Manager at the bunk should be able to see the trends of sale per employee
3. Employee comes and takes the shift.
4. After end of the shift, the emp will enter the sale data into the system
5. Per day one employee will enter only once into the sale data collection system

System requirements

1. System needs to support multiple tenants with data separation
2. 400+ concurrent max connections should be possible that will be doing add/view of records
3. Total 100 bunks need to be supported
    a. Each bunk has 8 to 10 employees
    b. Total of max 400 employees might add sale data concurrently. 10% buffer
    c. Sale data is entered only at the end of the shift.
4. Database requirement
    a. Should be able to support 400 concurrent writes into the table(of sale)
    b. Example: cassandra, redis, mongodb. If write fails, then it will retry adding that data, eventually it will bring the data to consistency.
5. In the user interface, while entering data, populate as many fields as possible in auto fashion.
6. GUI interfaces needed
    a. Dashboard
    b. Tanks, Inventory
    c. Pumps & Meters reading at opening/closing
    d. Sales log
7. UPI QR Integration
    a. Generate UPI QR codes for credit customer payments
    b. Verify if the amount credited at that time
8. Backup and restore
    a. Daily backup and storage
9. User login
    a. User name, role, PIN
10.

Pilot project runs

In 3 to 4 bunks we can do pilot project

1. Mobile app is needed
2. Desktop Dashboard needed
3. Cloud access for deployment

Failure modes and effect analysis (FMEA)

Feature level tests (FLTS)

Prompt given to AI

Task:

1. Create a document from this chat

2. In that document, Create a detailed user and system requirements

3. Create the detailed design document, that includes the user interfaces, database schema, tenant separation, and deployment strategies.
4. Consider the above user requirements given in the chat, and extract all of them into the above document
5. The design document need to include operations, financial, admin interfaces

WORKFLOW

1. Employee fuel delivery to customer, and how the reading is recorded
2. Booking keeping by management

Architecture

1. REST ful architecture
2. API based
    a. Get
    b. Put
    c. Delete
3. Each service will be deployed as a container/pods
    a. Controller
    b. Database (set of pods)
4.

02-March-2026
GUI Review

21:06 02-03-2026

Under "Staff & Allocation" tab
1. Shift manager, default employee directory, names are hard coded, when we change them, they need to reflect in the employee login drop down.
2. When editing the permissions, we only see what is not given. We also want to see, what permissions staff already has. The permissions which staff already has, should be shown as "checked" check-box. And we can edit, by unchecking or checking the permissions. The same need to be visible when i login into employee portal.
One of the reason, could be, we do not have persistence in terms of backend database. Once we have, then both should be referring to same tables.

3. Under "Pumps & Meters", when i change the pump1 nozzle-A to petrol and nozzle-B to Diesel, then under "staff & Allocation" we need to see the same changes

4. Under "Employee Directory", when i change name of the employee, everywhere, same name should reflect. For example if Raju K is changed to Bhima J, then we should see Bhima J, where ever applicable.

Under "Staff & Allocation"
6. Edit employee, and assign full day shift or multiple (morning,noon, night) shifts. currently, the GUI shows a list box, where only one selection is allowed. Make it as multiple selection.
7. When employee's shift is changed, then already allocated nozzles should be de-allocated, because that employee has now shifted to a different shift. Force manager to re-assigne employee to that particular nozzle

8. Under "Employee Portal", we always see, the first employee name only as "Active". Rather it should be showing which all employees have been logged in. In Desktop, it shows, only one logged in, in mobile it shows correctly all employees.
We need not have this employee portal, this can be removed.

9. Under "Reports" -> Create a new report "Employee Shift Report", along with attendance of the employee for a month. Here, show for every employee, what shifts have been done, and how many times.

10. Employee workflow
A) employee will come into fuel station, and then login into the employee portal using my own credentials.

B) Only when i am in the fuel station and login, only then it should take my attendance. Basically to avoid misuse of the login function, we need to have guardrails, checks and balances

C) i will verify that, the opening meter reading on the portal is same as that of actual meter reading on the pump

D) at the end of my shift, i will enter the closing meter reading and hand over shift to next person.

E) Manager is responsible for closure of my shift and handover to next person.

Under "Dashboard"
1. Extend the functionality of Shift status, to show which employees are LIVE for each of the shift.

11. Shift manager will record the Dip at the beginning of the day.

12. Employee records a sales. And that should reflect in the pumps total fuel availability. Check this functionality if its already this way. Provided a database is attached.

03/03/26

1) The expense register excel sheet description column needs junk value removal.
2) When switched from Employee portal the Dashboard of Admin is showing
3) In employee portal In Sales Menu Petrol is shown, whereas the Employee has been assigned Diesel nozzle.Closed
4) In Dashboard All employee who are logged in should be shown LIVE.
5) Employee login details need to be captured in Daily report
6) In Employee Shift Report this is missing “show for every employee, what shifts have been done”.
7) In DashBoard Credit Pending is not displayed correctly
8) Meter reading should be linked to Tank Inventory
9) In Employee Portal when Closing reading is added, it is not getting reflected in the respective reports and Pump and Meters

04/03/26
Employee portal > Whichever fuel type selected pump should be assigned(working in Premium).
Employee Portal >Overview> only cash is visible - Card Credit and UPI should be displayed
Employee Portal > Sales> Credit Management > Credit Limit Validation not happening (Eg Arvind 1000 limit crossing)

Employee Portal > Readings> Opening> Previous closing should be next login opening and should not be editable.
Admin Login> Sales> Pump column should display actual pump number add also nozzle column
Employee Portal>Closing Reading> Reading should not be edited.
Reports>Generate Full DSR Report not working
entire Employee Portal Sales menu should be rechecked
Purchase price should be less than selling price check is not there. Solution for this would be to update the price given in the record fuel purchase
Employee Shift Report > Remove> Shift done Column
In Dashboard> Cash In hand and credit pending should be removed instead put Petrol Diesel and Premium sold ltr today

1.====== RAW requirement

What did we achieve so far ?

- GUI has been confirmed, that it is according to the user requirements
- All GUI, functionality, users, everything has been verified. And fixed

Now, to proceed for a deployment alpha release to Fuel bunks.

- We need to persist data on the disk.
- Disk should be provided by the cloud deployment
- There should be interaction between the mobile/desktop browser and cloud server, where the data from browser is sent via restj-api to backend and it is written to a database.
- Database
    - We need to create database schema and required tables to store them
    - We need to ensure that it is secured
    - For initial testing purpose, where scalability or concurrency is not a concern, use file based database. Such as sql-lite for quick POC deployment as alpha release to test with actual end users and confirm functionality

===== AI prompt

<SACHIN to add actual AI prompt here >

==========

Expectation from this step

- The data entered has to be persisted
- Security
    - Idle browser login, should be logged out in X mins.
    - Browser close, logout
    - Ask AI to generate tests to do a security check against
        - DDOS attacks
        - SQL injection attacks
        - API failure, throttling ,

1. Rate Limiting / Throttling

- Controlling how many requests can be made in a time period (e.g., 100 requests per minute)
- Prevents overwhelming a server or API

2. Circuit Breaker Pattern

- Stops sending requests when failures reach a threshold
- Trips "open" → waits for cooldown → tries again ("half-open") → closes if successful
- Prevents cascading failures

3. Retry Logic with Backoff

- Automatically retrying failed requests
- **Exponential Backoff**: 1s → 2s → 4s → 8s delays between retries
- Prevents hammering a struggling server

4. Timeout Management

- Setting max wait time for API responses
- Prevents hanging indefinitely

5. Request Queueing

- Holding requests in a queue and processing them at a controlled rate
- Prevents sudden spikes

Combined, this is often called "Resilience Patterns" or **"Fault Tolerance Strategies"**.

Most modern libraries handle this automatically:

- **Axios** (JavaScript): retry interceptors, timeout settings
- **Fetch API**: custom retry logic needed
- **AWS SDK**: built-in retry and backoff
- **Node.js**: libraries like `retry`, `p-retry`, `async-retry`

Which API client/library are you using in your project?

SQL Injection Testing

1. **Manual Testing**
   - Inject SQL payloads in form fields: `' OR '1'='1`, `admin'--`, `'; DROP TABLE users;--`
   - Test all input fields: login forms, search bars, filters
2. **Automated Tools**
   - **SQLMap**: Automated SQL injection scanner
   - **Burp Suite Community**: Web vulnerability scanner
   - **OWASP ZAP**: Free security scanner
   - **Nikto**: Web server scanner
3. **Code Review**
   - Use parameterized queries/prepared statements (prevents injection)
   - Check for dynamic SQL concatenation (vulnerable)
   - Use ORM frameworks (reduce injection risk)
4. **Static Analysis Tools**
   - **SonarQube**: Code quality & security scanner
   - **Checkmarx**: SAST tool
   - **Bandit** (Python), **ESLint** (JS): Linters with security rules

DDoS Testing

1. **Load Testing** (Legal alternative to actual DDoS)
   - **Apache JMeter**: Simulate high traffic
   - **Locust**: Python-based load testing
   - **k6**: Modern load testing tool
   - Test: Can your server handle 1000+ concurrent requests?
2. **Infrastructure Hardening**
   - Enable **rate limiting** per IP
   - Use **WAF** (Web Application Firewall)
   - Deploy **CDN** (Cloudflare, Akamai) → distributes traffic
   - Implement **CORS** properly
   - Set **request timeouts**
3. **Monitoring & Alerts**
   - Monitor unusual traffic spikes
   - Set up alerts for abnormal patterns
   - Use **fail2ban** to auto-block suspicious IPs

General Security Tests

- **OWASP Top 10**: Test all 10 common vulnerabilities
- **Penetration Testing**: Hire professionals for thorough testing
- **HTTPS/TLS**: Ensure encryption in transit
- **Input Validation**: Whitelist allowed inputs
- **Authentication**: Test session management, password reset flows
- **API Security**: If you have APIs, test authentication, rate limiting

Quick Start Tools for Your Website

Since your site is a **Fuel Management PWA** (based on your workspace), also test:

- **Service Worker security** (shouldn't cache sensitive data)

- **Local Storage** (don't store tokens unencrypted)
- **API endpoint security** (if calling external APIs)

2. Next AI prompt

Next step, review the code generated by cloude.

- Security posture of the generated web function
- Run security tests to check for vulnerability – what tools are there to check this ?
- JWT token expiry is 12 hours ? should it be less or more ? what is correct for our use case ?

3. Template for AI prompt

Task:

1. Generate a prompt for the following context provided below.
2. Give the proper prompt that i should be using to get work done as described in context

Role:

1. I am a software engineer, and architect trying to create a web application that can help manage the Fuel station
2. I am also a security consultant to ensure that the web app deployed is free from all the vulnerabilities.

Context:

<COPY paste from above>

4. Deployment Options

Netlify **cannot** run a backend server or SQLite — it only hosts static files and short-lived serverless functions with no disk storage. SQLite needs a persistent filesystem to write to.

The simplest path that keeps it close to Netlify:

Option A — Split deploy (recommended):

- Frontend → Netlify (free, instant)
- Backend API + SQLite → Render.com (free, persistent disk)
- You already have both packages built and ready

Option B — Everything on Render.com:

- Single deploy, `render.yaml` already included in the zip
- Render serves both the frontend and the API from one Node.js process

Option C — Netlify + Turso (SQLite-in-the-cloud):

- Netlify Functions replace our `server.js`
- Turso replaces the SQLite file with a cloud SQLite database (same SQL syntax, free tier)
- More complex to set up, no real advantage for a POC

For your alpha/POC test with real users, **Render.com** is the fastest path:

1. Push the `fuelbunk-backend.zip` contents to GitHub
2. Go to render.com → New → Web Service
3. Connect the repo — it reads `render.yaml` automatically
4. Click **Deploy** — done in ~2 minutes
5. Free tier includes 1 GB persistent disk for the SQLite file

Would you like me to package a **split deploy version** (Netlify frontend + Render backend) or proceed with the single Render deploy?

Observations on 07/03/2026:

a) Multiple Superadmin logins are allowed as of now. → Not fixed
b) In Staff and allocation menu Add Shift should come first and then Add Employee should come. →Done

c) Add Shift Roaster functionality to the Staff and allocation page for admin login.
d) Attendance portal for admin or Owner user. →Done
e) While setting the pin for Employee, by default the cursor should high light “New 4-Digit Pin”--Done
f) In sales -> New Sales->Record New sale->
g) In Employee Portal Display Station Name at the TOP →Done
h) If the employee pin is not set. In the Employee login the Employees list should not be shown –Done

Observations on 10/3/2026:

a) Simplify this , too many icons –Done

b) Sort based on shift time.
c) Allocation should take employee names from the same day roster. –Done
d) Staff and allocation access should be given to the manager also.
e) On setting a mismatch PIN, the error should be updated as “PIN MISMATCH”
f) Review all the error messages and standardize them.
g) Employee enhancements
   i) Add Aadhar and employee ID for the newly added Employee
   ii) Employee ID should be unique key in the schema(Make it is u
   nique by using AI prompt)

h) Separate github project for test code is needed.
i) Install VScode, configure copilot in that.
j) Install Jenkins locally.
   i) Using selenium robot framework in python or robot lang, to create an automated test suite and perform the following
      1) Create 10 bunks each of different name
      2) Add employees under each bunk in the range of 5 to 15
      3) As a superadmin assign each employee
      4) Assign shift to each employee
      5) Then assign pump to employees
   ii) Login as employee, and start delivering petrol or diesel by each employee
   iii) This way, all employees in all the bunks start delivering petrol / diesel.

iv) Do this operation continuously for a given duration.

k) What are the limits in the current account
   i) CPU
   ii) Memory
   iii) Disk/storage : 500 mb
   iv) Network

l) Find out, which tool / utility is consuming disk space / table in postgres

m) If requirement is to support 100 bunks, and around 500 employees
   i) How much CPU, memory, disk and network bandwidth should be available in my website ? or account

Action Item on Krishna:

a) Which deployment platform is suitable for scale up (railway or render) and economical also – DONE
b) Currently we are using posgres database best suited for large application and economical also – Done

Currency Basis

- INR conversion uses 1 USD = ₹92 (today's public feeds show about ₹91.96–₹92.25 per USD).
- All amounts below are approximate and exclude taxes (like GST).

Cost Comparison (INR)

- Tenant-ready baseline (2 API replicas, 1 worker, Postgres 8GB, Redis 1GB, ~1TB outbound):
  - Railway: ₹25,000–₹33,000 / month
  - Render: ₹30,000–₹38,000 / month
- Growth profile (4–6 API replicas, 2–3 workers, Postgres 16GB + replica, Redis 5GB, ~3TB outbound):
  - Railway: ₹60,000–₹80,000 / month
  - Render: ₹80,000–₹1,00,000 / month
- Render's per-user plan fee is for your internal team members (dev/admin), not tenant employees.

Tenant-Ready Architecture

- CDN/WAF → Load Balancer → stateless API services (horizontal autoscaling).
- Separate Worker services for reports, reconciliation, and scheduled jobs via queue.
- PostgreSQL as primary datastore with strict tenant_id isolation, pooled connections, and backup/PITR.
- Redis for caching, rate-limit counters, queue broker, and transient session/token data.
- Object storage for receipts/invoices/files; keep app nodes stateless.
- Centralized logs + metrics + alerting (CPU, DB latency, queue lag, error rate).

Target Instance Size

- Start (fits 100+ tenants):
  - API: 2 × (1 vCPU, 2 GB)
  - Worker: 1 × (1 vCPU, 2 GB)

    - Postgres: 2 vCPU, 8 GB RAM, 100 GB disk
    - Redis: 1 GB
- Scale stage:
    - API: 4–6 replicas, move hot paths to 2 vCPU, 4 GB
    - Workers: 2–3 replicas (1–2 vCPU, 2–4 GB)
    - Postgres: 4 vCPU, 16 GB + 1 read replica (2 vCPU, 8 GB), 250–500 GB disk
    - Redis: 5 GB

Bottom Line

- For scaling maturity: Render is better.
- For lowest INR cost: Railway is usually cheaper.

Recommendation

- Adopt PostgreSQL as your primary database (continue with it).
- For your scale (100+ tenants, 5,000–10,000 employees total), it is the best balance of reliability, scalability, and cost.

Why PostgreSQL is the right fit

- Strong ACID transactions for fuel sales, expenses, credit ledgers, and reconciliations.
- Excellent relational querying/reporting (joins, aggregates, time-based analytics).
- Multi-tenant friendly with tenant_id isolation, Row-Level Security, and partitioning.
- Open-source (no license cost), so it is economical versus Oracle/SQL Server.
- Mature ecosystem: backups, PITR, read replicas, connection pooling, monitoring.

Quick comparison

- PostgreSQL: best overall for your workload and budget.
- MySQL: close second, but generally less flexible for complex analytics/reporting patterns.
- MongoDB: good for flexible documents, weaker fit for strict financial/transactional consistency.
- Enterprise DBs (Oracle/SQL Server): technically strong but usually much higher TCO.

Adoption model (recommended)

- Managed PostgreSQL (Postgres 16+) + PgBouncer + daily backups + PITR.
- Start with single primary; add 1 read replica when reporting load grows.
- Use one shared DB with strict tenant isolation first; move large tenants to dedicated DB only if needed.

If you want, I can give you a concrete PostgreSQL sizing plan (vCPU/RAM/storage, indexes, and pooling limits) for month-1 and month-6 in INR budget bands.